
MODULE *BlockDag*

In this specification we define notions on *DAGs* useful for DAG-based consensus protocols (which build *DAGs* of blocks)

EXTENDS *Digraph*, *FiniteSets*, *Sequences*, *Integers*

CONSTANTS

N The set of all nodes
 R The set of rounds
 $Leader(-)$ operator mapping each round to its leader

$Max(S) \triangleq \text{CHOOSE } x \in S : \forall y \in S : y \leq x$
 $Min(S) \triangleq \text{CHOOSE } x \in S : \forall y \in S : x \leq y$

For our purpose of checking safety and liveness, *DAG* vertices just consist of a node and a round.

$V \triangleq N \times R$ the set of possible *DAG* vertices
 $Node(v) \triangleq v[1]$
 $Round(v) \triangleq \text{IF } v = \langle \rangle \text{ THEN } 0 \text{ ELSE } v[2]$ accomodates $\langle \rangle$ as default value

Next we define how we order *DAG* vertices when we commit a leader vertice

$LeaderVertex(r) \triangleq \text{IF } r > 0 \text{ THEN } \langle Leader(r), r \rangle \text{ ELSE } \langle \rangle$
 $IsLeader(v) \triangleq LeaderVertex(Round(v)) = v$
 $Genesis \triangleq \langle \rangle$

$OrderSet(S)$ arbitrarily order the members of the set S . Note that, in TLA+, CHOOSE is deterministic but arbitrary choice, *i.e.* $\text{CHOOSE } e \in S : \text{TRUE}$ is always the same e if S is the same

RECURSIVE $OrderSet(-)$

$OrderSet(S) \triangleq \text{IF } S = \{\} \text{ THEN } \langle \rangle \text{ ELSE}$
 $\text{LET } e \triangleq \text{CHOOSE } e \in S : \text{TRUE}$
 $\text{IN } Append(OrderSet(S \setminus \{e\}), e)$

$PreviousLeader(dag, r) \triangleq$

$\text{IF } \exists l \in Vertices(dag) : IsLeader(l) \wedge Round(l) < r$
 THEN

$\text{CHOOSE } l \in Vertices(dag) :$

$\wedge IsLeader(l) \wedge Round(l) < r$

$\wedge \forall v \in Vertices(dag) : IsLeader(v) \wedge Round(v) < r \Rightarrow Round(v) \leq Round(l)$

ELSE

$\langle \rangle$

RECURSIVE $Linearize(-, -)$

$Linearize(dag, l) \triangleq \text{IF } Vertices(dag) = \{\langle \rangle\} \text{ THEN } \langle \rangle \text{ ELSE}$

$\text{LET } dagOfL \triangleq SubDag(dag, \{l\})$

$prevL \triangleq PreviousLeader(dagOfL, Round(l))$

$dagOfPrev \triangleq SubDag(dag, \{prevL\})$

$remaining \triangleq Vertices(dagOfL) \setminus Vertices(dagOfPrev)$
 IN $Linearize(dagOfPrev, prevL) \circ OrderSet(remaining \setminus \{l\}) \circ \langle l \rangle$
 technically, we should use a topological sort instead of $OrderSet(-)$
